

6. Multimodal video analysis application

6. Multimodal video analysis application

1. Concept Introduction
 - 1.1 What is "Video Analysis"?
 - 1.2 Implementation Principle Overview
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
 2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring the Offline Large Model
 - 3.1.1 Configuring the LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuration model interface (`large_model_interface.yaml`)
 - 3.2 Starting and Testing the Function

1. Concept Introduction

1.1 What is "Video Analysis"?

In the `largemodel` project, the **multimodal video analysis** feature enables a robot to process a video and summarize its core content, describe key events, or answer specific questions about the video in natural language. This allows the robot to leap from understanding only static images to understanding the dynamic and temporal world.

The core tool for this feature is `**analyze_video``. When a user provides a video file and asks a question (for example, "Summarize what this video says"), the system invokes this tool to process and analyze the video and return a textual response from the AI.

1.2 Implementation Principle Overview

The core challenge of offline video analysis lies in how to efficiently process video data containing hundreds or thousands of frames. A popular implementation principle is as follows:

1. **Keyframe Extraction:** First, the system does not process every frame of the video. Instead, it uses algorithms (such as scene boundary detection or fixed-time sampling) to extract the most representative **keyframes**. This significantly reduces the amount of data required for processing.
2. **Image Encoding:** Each extracted keyframe is fed into a visual encoder, similar to visual understanding applications, to be converted into a digital vector containing image information.
3. **Temporal Information Fusion:** This is the most significant difference from single-image understanding. The model needs to understand the temporal order between these keyframes. Typically, a recurrent neural network (RNN) or Transformer model is used to fuse the vectors of all keyframes to form a "memory" vector that represents the dynamic content of the entire video.
4. **Question Answering and Generation:** The user's text question is encoded and then cross-modally fused with this "memory" vector. Finally, the language model generates a summary

of the entire video or an answer to a specific question based on this fused information.

Simply put, it condenses a video into a few key images and their sequence, allowing the user to understand the entire story like a comic strip and answer related questions.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `analyze_video` function in this file defines the tool's execution flow.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def analyze_video(self, args):
        """
        Analyze video file and provide content description.
        分析视频文件并提供内容描述。

        :param args: Arguments containing video path.
        :return: Dictionary with video description and path.
        """
        self.node.get_logger().info(f"Executing analyze_video() tool with args:
{args}")
        try:
            video_path = args.get("video_path")
            # ... (智能路径回退机制)

            if video_path and os.path.exists(video_path):
                # ... (构建Prompt)

                # Use a fully isolated, one-time context for video analysis to
                ensure a plain text description. / 使用完全隔离的一次性上下文进行视频分析，以确保获得纯文
                本描述。

                simple_context = [{
                    "role": "system",
                    "content": "You are a video description assistant. ..."
                }]

                result = self.node.model_client.infer_with_video(video_path,
                    prompt, message=simple_context)

                # ... (处理结果)
                return {
                    "description": description,
                    "video_path": video_path
                }
            # ... (错误处理)
```

2. Model Interface Layer and Frame Extraction (`largemodel/utils/large_model_interface.py`)

The functions in this file are responsible for processing video files and passing them to the underlying model.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_video(self, video_path, text=None, message=None):
        """Unified video inference interface. / 统一的视频推理接口。"""
        # ... (Prepare Message)
        try:
            # Determine which specific implementation to call based on
            self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
video_path=video_path)
            # ... (The logic of other online platforms)
            # ...
            return {'response': response_content, 'messages': self.messages.copy()}

        def _extract_video_frames(self, video_path, max_frames=5):
            """Extract keyframes from a video for analysis. / 从视频中提取关键帧用于分
析。"""
            try:
                import cv2
                # ... (Video reading and frame interval calculation)
                while extracted_count < max_frames:
                    # ... (Looping through video frames)
                    if frame_count % frame_interval == 0:
                        # ... (Save the frame as a temporary image)
                        frame_base64 = self.encode_file_to_base64(temp_path)
                        frame_images.append(frame_base64)
                    # ...
                return frame_images
            # ... (Exception handling)
```

Code Analysis

The implementation of video analysis is more complex than image analysis. It requires a key preprocessing step at the model interface layer: frame extraction.

1. Tools Layer (`tools_manager.py`):

- The `analyze_video` function is the entry point for video analysis. Its responsibilities are clear: accept a video file path and construct a prompt to request a description of the video content.
- It initiates the analysis process by calling the `self.node.model_client.infer_with_video` method. Like the visual understanding tool, it is completely agnostic about the underlying model details and only handles passing the "video file" and "analysis instructions."

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_video` function is the scheduler that connects the upper-level tools with the underlying model. It dispatches tasks to the corresponding implementation functions based on the platform configuration (`self.llm_platform`).
- Unlike processing single images, processing videos requires an additional step. The `_extract_video_frames` method demonstrates the general logic for implementing this step: it uses the `cv2` library to read the video and extract several keyframes (by default, five).
- Each extracted frame is treated as a separate image and typically encoded as a Base64 string.
- Ultimately, a request containing multiple frame image data and analysis instructions is sent to the main model. The model performs a comprehensive analysis of these consecutive images to generate a description of the entire video content.
- This "video-to-multiple-images" preprocessing is completely encapsulated in the model interface layer and is transparent to the tool layer.

In summary, the general process of video analysis is: `ToolsManager` initiates an analysis request -> `model_interface` intercepts the request and calls `_extract_video_frames` to decompose the video file into multiple keyframe images -> `model_interface` sends these images, along with analysis instructions, to the corresponding model platform according to the configuration -> the model returns a comprehensive description of the video -> the results are finally returned to `ToolsManager`. This design ensures the stability and versatility of upper-layer applications.

3. Practical Operations

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (yahboom.yaml)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in Terminal:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. Modify/Confirm `llm_platform`:

```
model_service:                                     #Model server node parameters
  ros__parameters:
    language: 'en'                                  #Large Model Interface Language
    useolnetts: True                                #This item is invalid in text mode
    and can be ignored

    # Large model configuration
    llm_platform: 'ollama'                          # Key: Make sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 Configuration model interface (large_model_interface.yaml)

This file defines which visual model to use when the platform is selected as `ollama`.

1. Open the file in a terminal.

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama related configuration

```
#.....  
## 离线大模型 (Offline Large Language Models)  
# Ollama Configuration  
ollama_host: "http://localhost:11434" # Ollama server address  
ollama_model: "llava" # Key: Change this to the multimodal model you  
downloaded, such as "llava"  
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Starting and Testing the Function

Note: Due to performance limitations, this example cannot be run on the Jetson Orin Nano 4GB. To experience this feature, please refer to the corresponding section in <Online Large Model (Voice Interaction)>

1. Prepare a video file:

Place the video file to be tested in the following path:

```
/home/jetson/yahboom_ws/src/largemodel/resources_file/analyze_video
```

Then name the video `test_video.mp4`

2. Start the `largemodel` main program:

Open a terminal and run the following command

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake up:** Say "Hi,yahboom" into the microphone.
- **Dialogue:** After the speaker responds, you can say, "Analyze the video content."
- **Observe the log:** In the terminal running the `launch` file, you should see the following:
 1. The ASR node recognizes your question and prints it.
 2. The `model_service` node receives the text, calls the LLM, and prints the LLM's response.
- **Listen for the answer:** After a while, you should hear the answer from the speaker.